

Trabajo Final Integrador

FixHub

Sistema Integral de Gestión para Talleres y Servicios Técnicos



Grupo 186

Integrantes:

- Damián Nogueira
- Gabriel Etchegoyen
- Manuel Galarza

Repositorio: <https://github.com/damiannogueira/fixhub>



Índice

Identificación del problema y propuesta de solución	3
1. Contexto del problema	3
2. Problema identificado	4
3. Actores involucrados	5
4. Flujo de trabajo actual	6
5. Necesidades detectadas	8
6. Impacto del problema	9
7. Propuesta de solución	11
8. Valor agregado	11
9. Planteamiento formal del problema	12
10. Validación inicial	13
Informe de Definición del Stack Tecnológico	14
1. Introducción	14
2. Descripción General del Stack	14
3. Justificación de las Elecciones Tecnológicas	15
Propuesta y Plan de trabajo	22
1. Refinamiento de la Idea y Propuesta de Valor	22
2. Análisis de Competencia y Diferenciación	24
3. Objetivo general y específicos	25
4. Resumen de viabilidad técnica, operativa y temporal	30
5. Conclusiones finales	30

Identificación del problema y propuesta de solución

1. Contexto del problema

En la actualidad, muchos talleres y servicios técnicos de pequeño y mediano porte —como reparación de celulares, electrónica, computación, electrodomésticos, o mantenimiento en general— gestionan su operación diaria utilizando herramientas no integradas: agendas de papel, planillas de cálculo, mensajes de WhatsApp y llamadas telefónicas.

La idea de FixHub surgió a partir de una problemática real planteada por uno de los integrantes del equipo, observada en su actividad dentro de un servicio técnico electromecánico automotriz. En ese contexto, diferentes tareas de gestión se realizan mediante contacto directo con los clientes por WhatsApp, llamadas telefónicas o de forma presencial, mientras que otros controles internos, como el stock de repuestos, se llevan mediante planillas de cálculo. A partir de esta situación, el equipo identificó que la fragmentación de la información y la falta de integración entre los distintos procesos no es una problemática exclusiva del rubro automotor, sino que puede presentarse también en otros tipos de servicios técnicos.

Cada reparación implica un ciclo que comienza con la recepción de un equipo, sigue con su diagnóstico, la cotización de repuestos y mano de obra, la reparación propiamente dicha, y finaliza con la entrega al cliente. A lo largo de este proceso, intervienen distintas personas: quien recibe el equipo, quien lo repara, quien administra los repuestos, y el propio cliente que espera novedades.

Cuando cada una de estas actividades se registra en herramientas separadas, la información queda fragmentada. No hay una vista unificada que permita saber rápidamente en qué estado está una reparación, qué técnico la tiene asignada, qué repuestos se utilizaron, o si el cliente ya aprobó el presupuesto.

Esta situación no implica que el taller "no funcione", sino que funciona con ineficiencias toleradas: se pierde tiempo buscando información, se depende de la memoria de los empleados, se duplican registros, y el cliente recibe información de manera inconsistente o tardía.

Por lo tanto, el problema no consiste simplemente en que "falta una aplicación para talleres", sino en que la información necesaria para gestionar el ciclo de reparaciones se encuentra dispersa, se gestiona manualmente y no fluye de manera integrada entre las personas y etapas del proceso.

2. Problema identificado

Los talleres y servicios técnicos que gestionan sus reparaciones utilizando herramientas separadas (papel, planillas de cálculo, mensajería) enfrentan las siguientes dificultades:

Falta de visibilidad del estado de las reparaciones: No existe una manera rápida y centralizada de conocer en qué etapa se encuentra una orden de trabajo. Para saber el estado de una reparación, es necesario preguntar directamente al técnico o buscar en registros dispersos.

Información fragmentada entre actores: El recepcionista registra el equipo y el problema informado; el técnico realiza el diagnóstico y registra las tareas; el administrador gestiona repuestos y presupuestos. Cada uno trabaja con su propia herramienta o registro, sin que exista un punto de integración.

Dificultad para mantener relacionada la información: Cliente, equipo, orden de trabajo, diagnóstico, presupuesto, repuestos utilizados e historial de reparaciones deberían estar vinculados. En el modelo actual, estas relaciones se pierden o se reconstruyen manualmente.

Gestión manual de presupuestos y aprobaciones: El presupuesto se prepara sin tener una vista clara del stock de repuestos ni del costo real de mano de obra. La aprobación del cliente se gestiona por llamada o mensaje, sin quedar registrada formalmente en el sistema.

Control de inventario inconsistente: Los repuestos se utilizan en las reparaciones, pero no siempre se descuentan del stock de manera sistemática. Esto genera faltantes no previstos y dificultad para saber qué insumos están disponibles.

Falta de historial de reparaciones: No se conserva un registro histórico de las reparaciones realizadas a un mismo equipo o cliente. Esto impide identificar problemas recurrentes o mantener un seguimiento adecuado.

Comunicación con el cliente poco fluida: El cliente debe llamar o enviar un mensaje para conocer el estado de su reparación, y no siempre recibe una respuesta clara o actualizada.

Ausencia de información para la gestión: Sin datos sistematizados, es difícil obtener indicadores como: cantidad de reparaciones por técnico, tiempos promedio de entrega, repuestos más utilizados, o ingresos generados.

Como resultado, el taller funciona con un costo oculto de tiempo y esfuerzo dedicado a tareas administrativas y de coordinación, que podrían reducirse si la información estuviera centralizada y automatizada.

3. Actores involucrados

Para comprender quién interactúa con el proceso y con el sistema, se identifican los siguientes actores:

Actor	Rol	Necesidades principales	Limitaciones
Cliente	Persona que lleva un equipo al taller para su reparación.	Consultar el estado de su reparación. Conocer el diagnóstico y presupuesto. Aprobar o rechazar el presupuesto. Consultar los avances de la reparación.	Puede no tener acceso a internet en todo momento. Puede tener dificultades técnicas para usar una plataforma digital.
Recepcionista	Persona que recibe los equipos, registra clientes y gestiona la entrada y salida de órdenes.	Registrar clientes y equipos de manera rápida. Crear órdenes de trabajo. Registrar el problema informado por el cliente. Gestionar la recepción y entrega de equipos.	Puede tener alta carga de trabajo en horarios pico. Puede no contar con experiencia técnica profunda.
Técnico	Persona que realiza el diagnóstico, la reparación y las tareas técnicas.	Consultar las órdenes asignadas. Registrar diagnóstico y tareas realizadas. Indicar repuestos necesarios o utilizados. Actualizar el estado de la orden según las etapas permitidas.	Puede tener limitaciones de tiempo para registrar información detallada. Puede trabajar en múltiples órdenes simultáneamente.
Administrador	Persona responsable de la gestión operativa y comercial del taller.	Gestionar usuarios y roles. Administrar el inventario de repuestos. Consultar todas las órdenes y su estado. Visualizar métricas y reportes de gestión.	Puede necesitar información consolidada para la toma de decisiones. Puede tener responsabilidades fuera del sistema.

La identificación de actores permite comprender que el sistema debe adaptarse a distintos perfiles de usuario, con diferentes niveles de acceso y funcionalidades, y debe resolver necesidades específicas de cada rol dentro del proceso de reparación.

En servicios técnicos pequeños, una misma persona puede desempeñar más de una función dentro del negocio. Por ejemplo, el propietario puede cumplir simultáneamente tareas de administración y tareas técnicas. Por este motivo, el sistema deberá contemplar que un usuario pueda asumir distintas responsabilidades según la organización del servicio técnico.

4. Flujo de trabajo actual

El proceso que sigue actualmente un taller o servicio técnico para gestionar una reparación, utilizando herramientas no integradas, puede describirse de la siguiente manera:

4.1. Recepción del equipo

- El cliente llega al taller con un equipo que presenta una falla o requiere mantenimiento.
- El recepcionista registra manualmente en una planilla de papel o planilla de cálculo los datos del cliente, el equipo y el problema informado.
- El recepcionista asigna un número de orden de trabajo de manera manual o secuencial.
- El equipo es etiquetado físicamente y enviado al área de diagnóstico.

4.2. Diagnóstico

- El técnico recibe el equipo y realiza el diagnóstico para identificar la falla.
- El técnico registra en su propia herramienta (papel, planilla, o simplemente mentalmente) el diagnóstico y las tareas necesarias.
- Si se detectan repuestos necesarios, el técnico consulta al administrador o verifica físicamente el stock disponible.

4.3. Presupuesto y aprobación

Con el diagnóstico realizado, el técnico o el recepcionista prepara un presupuesto que incluye mano de obra y repuestos.

- El presupuesto se comunica al cliente por teléfono o mensaje de WhatsApp.
- El cliente manifiesta su aprobación o rechazo verbalmente.
- La aprobación no queda registrada formalmente en ningún sistema.

4.4. Reparación

- Si el presupuesto es aprobado, el técnico procede con la reparación.
- El técnico utiliza los repuestos necesarios y registra su consumo en su propia planilla (o no lo registra).
- El stock físico de repuestos no se actualiza automáticamente.

4.5. Entrega y seguimiento

Una vez finalizada la reparación, el técnico comunica verbalmente o por escrito al recepcionista que la orden está lista.

- El recepcionista actualiza manualmente la planilla con el estado "finalizado".
- El cliente debe llamar o enviar un mensaje para consultar el estado de su reparación.
- Cuando el cliente retira el equipo, el recepcionista registra la entrega en la planilla.
- El historial de reparaciones no se conserva de manera estructurada para futuras consultas.

4.6. Análisis del flujo

El flujo actual evidencia múltiples tareas manuales y repetitivas:

- El registro de clientes, equipos y órdenes se realiza en herramientas separadas (papel, planillas, mensajes).
- La comunicación entre actores (recepcionista, técnico, administrador, cliente) depende de canales no integrados (WhatsApp, llamadas, comunicación verbal).
- El diagnóstico, el presupuesto y la aprobación no quedan registrados formalmente.
- El cliente no tiene visibilidad del estado de su reparación sin realizar una consulta activa.
- No se genera información histórica que permita análisis o toma de decisiones.

Como consecuencia, el taller dedica tiempo a tareas administrativas y de coordinación que podrían reducirse o eliminarse si la información estuviera centralizada y automatizada.

5. Necesidades detectadas

A partir del análisis del problema y el flujo de trabajo actual, se han identificado las siguientes necesidades:

5.1. Necesidades del cliente

- Consultar el estado de su reparación sin necesidad de llamar o enviar mensajes.
- Conocer el diagnóstico y el presupuesto de manera clara y accesible.
- Aprobar o rechazar el presupuesto de forma digital y con registro formal.
- Consultar los cambios de estado y avances de su reparación.

5.2. Necesidades del recepcionista

- Registrar clientes, equipos y órdenes de manera rápida y centralizada.
- Asociar automáticamente cliente, equipo y orden de trabajo.
- Visualizar el estado actual de todas las órdenes.
- Gestionar la recepción y entrega de equipos con registros claros.

5.3. Necesidades del técnico

- Consultar las órdenes de trabajo asignadas de manera clara.
- Registrar el diagnóstico y las tareas realizadas.
- Indicar repuestos necesarios y consumidos.
- Actualizar el estado de la orden según las etapas permitidas.

5.4. Necesidades del administrador

- Gestionar usuarios y roles con diferentes permisos.
- Administrar el inventario de repuestos.
- Visualizar todas las órdenes y su estado.
- Generar indicadores y reportes para la toma de decisiones.

5.5. Necesidades comunes

- Centralizar toda la información del taller en un único sistema.
- Automatizar la actualización de stock al consumir repuestos.
- Conservar un historial completo de reparaciones por cliente y equipo.
- Simplificar la comunicación con el cliente a través del sistema.

6. Impacto del problema

El principal impacto se encuentra en el **tiempo y esfuerzo** necesarios para gestionar el ciclo de reparaciones sin herramientas integradas.

A partir del análisis del flujo de trabajo actual, clasificamos los siguientes impactos:

6.1. Impacto en tiempo

- El recepcionista debe registrar manualmente los mismos datos en múltiples herramientas (papel, planilla, mensajes).
- El técnico debe buscar información sobre la orden, el equipo y el cliente en registros dispersos.
- El administrador debe consultar a distintas personas para conocer el estado de una reparación o el stock disponible.
- El cliente debe realizar consultas activas por teléfono o mensaje para conocer el estado de su reparación.

6.2. Impacto en calidad de la información

- La información registrada puede ser inconsistente o incompleta al depender de registros manuales.
- Las relaciones entre cliente, equipo y orden de trabajo no están formalizadas.
- El historial de reparaciones no se conserva de manera estructurada.

6.3. Impacto en la gestión

- El control de stock no es confiable porque los consumos no se registran sistemáticamente.
- No se dispone de información agregada para la toma de decisiones (indicadores, reportes).
- El conocimiento del taller depende de la memoria de los empleados, lo que genera riesgos ante su ausencia.

6.4. Estimación preliminar del impacto

A partir de los impactos detectados, se realizó una estimación preliminar de los tiempos que pueden demandar algunas tareas cuando la gestión se realiza sin un sistema centralizado. Estos valores son aproximados y deberán contrastarse posteriormente durante el relevamiento con otros talleres y servicios técnicos.

Actividad	Estimación
Tiempo promedio para registrar un cliente + equipo + orden	5-10 minutos (manual)
Tiempo promedio para consultar el estado de una orden	2-5 minutos (preguntar/buscar)
Tiempo promedio para gestionar un presupuesto y aprobación	10-15 minutos (mensajes/llamadas)
Tiempo promedio para actualizar stock	5-10 minutos (manual, al final del día)
Órdenes promedio por día en un taller pequeño	5-10 órdenes
Tiempo estimado perdido en tareas administrativas por día	1-2 horas

Observaciones: Con la implementación del sistema se espera reducir parte del tiempo destinado a tareas administrativas y de coordinación, principalmente mediante la centralización de la información y la automatización de tareas repetitivas, como la actualización de stock y el seguimiento del estado de las órdenes. El impacto real podrá evaluarse una vez que el sistema sea implementado y probado.

7. Propuesta de solución

Se propone desarrollar un Sistema Integral de Gestión para Talleres y Servicios Técnicos, una aplicación web que permita administrar de forma integrada el ciclo completo de trabajo de un taller o servicio técnico, desde la recepción del equipo hasta su entrega al cliente.

El sistema centralizará la información de:

- Clientes
- Equipos
- Órdenes de trabajo
- Diagnósticos
- Presupuestos
- Repuestos
- Stock
- Historial de acciones
- Seguimiento del cliente
- Reportes básicos de gestión (métricas)

El objetivo no es simplemente digitalizar una planilla, sino utilizar la información para controlar el proceso completo, aplicando reglas de negocio que automaticen tareas, mantengan la integridad de los datos y mejoren la comunicación entre todos los actores.

8. Valor agregado

El valor del sistema propuesto no consiste en "digitalizar" una planilla o reemplazar el papel por una pantalla, sino en transformar la gestión del taller mediante la centralización y automatización de la información.

8.1. Valor agregado para el cliente

- Visibilidad del estado de su reparación sin necesidad de llamar.
- Acceso al diagnóstico y presupuesto de manera clara.
- Aprobación digital del presupuesto, con registro formal.
- Tranquilidad y confianza al saber que su reparación está siendo gestionada de manera ordenada.

8.2. Valor agregado para el recepcionista

- Reducción del tiempo de registro al centralizar los datos en un único sistema.
- Asociación automática entre cliente, equipo y orden de trabajo.
- Visibilidad del estado de todas las órdenes en un solo lugar.

8.3. Valor agregado para el técnico

- Acceso rápido a las órdenes asignadas.
- Registro estructurado de diagnóstico y tareas realizadas.
- Control de repuestos necesarios y consumidos.

8.4. Valor agregado para el administrador

- Control total del inventario con actualización automática de stock.
- Visibilidad completa de todas las órdenes y su estado.
- Indicadores y reportes para la toma de decisiones.
- Historial de reparaciones por cliente y equipo.

- Cuadro resumen del valor agregado

Aspecto	Sin sistema	Con sistema
Registro de información	Manual, en múltiples herramientas	Centralizado, en un solo lugar
Estado de las órdenes	Se consulta preguntando	Visible en tiempo real
Control de stock	Manual, no confiable	Automático, actualizado
Comunicación con el cliente	Por teléfono/mensaje, sin registro	A través del sistema, con registro
Historial de reparaciones	No se conserva	Historial completo por cliente y equipo
Información para gestión	No disponible	Indicadores y reportes disponibles

9. Planteamiento formal del problema

Los talleres y servicios técnicos que gestionan sus reparaciones utilizando herramientas separadas como papel, planillas de cálculo, mensajes de WhatsApp y llamadas telefónicas enfrentan dificultades para:

- Conocer rápidamente el estado actual de una reparación.
- Saber qué técnico tiene asignado un trabajo.
- Mantener relacionados cliente, equipo y orden de trabajo.
- Registrar diagnósticos y tareas realizadas.
- Gestionar presupuestos y su aprobación por parte del cliente.
- Conocer qué repuestos se utilizaron en cada reparación.
- Mantener actualizado el stock de repuestos.
- Conservar un historial de las acciones realizadas.
- Informar al cliente sobre el avance de su reparación.
- Obtener información sobre tiempos, reparaciones y consumo de repuestos.

El sistema propuesto centraliza toda esta información, automatiza tareas manuales y permite que el taller funcione con mayor eficiencia, transparencia y control sobre su operación.

10. Validación inicial

El problema identificado parte de una situación real observada en un servicio técnico electromecánico automotriz, donde distintos procesos de gestión se realizan mediante herramientas y canales no integrados. Esta experiencia funcionó como punto de partida para el análisis realizado por el equipo y para la definición de una problemática más general aplicable a otros tipos de servicios técnicos.

Para validar que esta problemática también se presenta en otros contextos y comprobar que la solución propuesta puede resultar útil más allá del caso inicial, se propone realizar consultas a talleres y servicios técnicos de distintos rubros.

Sobre el proceso actual:

- ¿Cómo registran clientes, equipos y reparaciones?
- ¿Qué herramientas utilizan (papel, planillas, WhatsApp, software)?
- ¿Qué dificultades encuentran en el proceso?

Sobre el impacto:

- ¿Cuánto tiempo dedican diariamente a tareas administrativas?
- ¿Con qué frecuencia tienen problemas con el control de stock?
- ¿Cómo se informa al cliente sobre el estado de su reparación?

Sobre la solución:

- ¿Qué funcionalidades consideran esenciales en un sistema de gestión?
- ¿Utilizan actualmente algún software? En caso afirmativo, ¿qué problemas o limitaciones les genera?

Estas preguntas permitirán contrastar la problemática con otros contextos, evaluar su relevancia y conocer el interés de potenciales usuarios en la solución propuesta.

Informe de Definición del Stack Tecnológico.

1. Introducción

El presente informe tiene como objetivo definir y justificar el stack tecnológico a utilizar en el desarrollo del Sistema Integral de Gestión para Talleres y Servicios Técnicos denominado FixHub, una aplicación web que permita administrar de forma integrada el ciclo completo de trabajo de un taller o servicio técnico, desde la recepción del equipo hasta su entrega al cliente.

La selección de cada tecnología se ha realizado siguiendo los criterios establecidos en la guía de la unidad, priorizando la adecuación al problema, la madurez de las tecnologías, la experiencia del equipo y la viabilidad del proyecto.

2. Descripción General del Stack

Capa	Tecnología	RoI
Frontend	HTML + CSS + JavaScript + TypeScript	Interfaz de usuario, lógica de presentación y tipado estático
Backend	Java + Spring Boot (Maven)	API REST, lógica de negocio, seguridad y persistencia
Base de Datos	MySQL	Persistencia de datos relacionales
ORM	Spring Data JPA (Hibernate)	Mapeo objeto-relacional y gestión de entidades
Autenticación	Spring Security + JWT	Seguridad y control de acceso basado en roles
Frontend Build Tool	Vite	Empaquetado y desarrollo del frontend
Despliegue	Vercel/Netlify (Frontend) + Render/Railway (Backend) + MySQL administrado	Entorno de ejecución en la nube
Control de Versiones	Git + GitHub	Gestión del código fuente y trabajo colaborativo

3. Justificación de las Elecciones Tecnológicas

3.1. Principio Fundamental

El stack elegido ha sido seleccionado teniendo en cuenta las experiencias previas del equipo y la adecuación de las tecnologías a las necesidades del proyecto.

Los integrantes han utilizado Java y Spring Boot a lo largo de su formación, lo que les ha permitido desarrollar proyectos con lógica de negocio compleja, APIs REST y persistencia de datos. MySQL ha sido la base de datos relacional utilizada en múltiples trabajos prácticos, por lo que el equipo cuenta con experiencia en modelado de datos, consultas SQL y gestión de transacciones.

En el frontend, el equipo posee experiencia en HTML, CSS y JavaScript, tecnologías fundamentales del desarrollo web. Adicionalmente, han incorporado TypeScript como una extensión de JavaScript que aporta tipado estático, lo que permite detectar errores en tiempo de desarrollo y mejorar la mantenibilidad del código. Esta decisión se alinea con el principio de elegir tecnologías conocidas, ya que TypeScript complementa los conocimientos existentes de JavaScript sin requerir un cambio de paradigma.

Esto permite reducir la curva de aprendizaje, escribir y revisar código más rápidamente, y conocer las limitaciones desde el inicio, tal como indica la guía.

3.2. Naturaleza del problema

- Backend: Java + Spring Boot

La naturaleza del problema exige un backend robusto con las siguientes capacidades:

1. Lógica de negocio compleja

El sistema debe gestionar múltiples entidades interrelacionadas:

- Clientes
- Equipos
- Órdenes de trabajo
- Diagnósticos
- Presupuestos
- Repuestos
- Stock
- Historial
- Usuarios y roles

Además, debe implementar reglas de negocio que incluyen:

- Transiciones de estado de las órdenes:
(Recibido → En diagnóstico → Pendiente de aprobación → Aprobado → En reparación → Finalizado → Listo para retirar → Entregado, contemplando también el rechazo del presupuesto y la cancelación de la orden cuando corresponda).
- Una orden que requiera presupuesto no podrá pasar al estado “En reparación” hasta que dicho presupuesto haya sido aprobado por el cliente.
- Validaciones de stock al consumir repuestos
- Cálculo automático de totales de presupuesto (mano de obra + repuestos)
- Control de permisos según el rol del usuario
- Registro de historial de cambios en las órdenes

2. API REST

Spring Boot permite construir APIs REST de manera eficiente y estructurada, con soporte para:

- Controladores REST con anotaciones (@RestController, @RequestMapping)
- Validación de datos con Bean Validation (@Valid, @NotNull, etc.)
- Manejo de excepciones global
- Documentación automática con SpringDoc OpenAPI

3. Seguridad y autenticación

Spring Security es el estándar de facto para seguridad en aplicaciones Java. Permite implementar:

- Autenticación basada en JWT
- Control de acceso basado en roles (Administrador, Recepcionista, Técnico, Cliente)
- Protección de endpoints según el rol del usuario

4. Persistencia y transacciones

Spring Data JPA con Hibernate simplifica la interacción con la base de datos, ofreciendo:

- Mapeo de entidades a tablas mediante anotaciones (@Entity, @OneToMany, etc.)
- Repositorios con métodos CRUD automáticos
- Soporte para transacciones ACID con @Transactional
- Consultas personalizadas con JPQL o SQL nativo

Conclusión: Java + Spring Boot es una elección adecuada para un sistema de gestión con estas reglas de negocio, múltiples actores y operaciones transaccionales.

- Frontend: HTML + CSS + JavaScript + TypeScript

Interfaz de usuario interactiva:

El sistema requiere una interfaz funcional que permita a los distintos tipos de usuarios (repcionista, técnico, administrador, cliente) interactuar con las funcionalidades según su rol. HTML, CSS y JavaScript permiten construir interfaces dinámicas sin la dependencia de frameworks externos.

Tipado estático con TypeScript:

TypeScript se incorpora como una capa adicional sobre JavaScript que aporta:

- Detección temprana de errores: Los errores de tipo se identifican en tiempo de compilación, no en tiempo de ejecución.
- Mejor mantenibilidad: El código es más fácil de entender y modificar, especialmente cuando el proyecto crece.
- Autocompletado y documentación: Los editores de código ofrecen mejor soporte con TypeScript.
- Tipado de respuestas de la API: Los objetos que llegan desde el backend (clientes, órdenes, repuestos) pueden ser tipados, evitando errores como acceder a propiedades inexistentes.

La combinación de JavaScript y TypeScript permite al equipo trabajar con la flexibilidad de JavaScript cuando es necesario, y con la seguridad de TypeScript en partes críticas del código.

Conclusión: HTML + CSS + JavaScript + TypeScript es una elección equilibrada que combina la simplicidad de las tecnologías fundamentales con las ventajas del tipado estático.

3.3. Tipo de Bases de Datos

- Base de Datos: MySQL (SQL)

El sistema necesita persistir información con una estructura definida y relaciones claras entre entidades. Se identifican las siguientes relaciones:

Cliente → Equipo (uno a muchos)
Equipo → Orden de trabajo (uno a muchos)
Orden de trabajo → Diagnóstico (uno a uno)
Orden de trabajo → Presupuesto (uno a uno)
Presupuesto → DetallePresupuesto (uno a muchos)
DetallePresupuesto → Repuesto (muchos a uno)
Repuesto → Stock (uno a uno)
Orden de trabajo → Historial (uno a muchos)
Orden de trabajo → Técnico (muchos a uno)
Orden de trabajo → Estado (muchos a uno)

Además, se requiere integridad transaccional (ACID) especialmente en operaciones como:

- Registrar el consumo de un repuesto en una reparación
- Descontar directamente del stock
- Asegurar que no quede un movimiento registrado sin actualizar el inventario

Estas características hacen que una base de datos relacional sea la opción más adecuada. MySQL ha sido seleccionado porque:

Se integra fácilmente con Spring Data JPA a través del driver JDBC.
Existen servicios administrados en la nube que simplifican el despliegue.

Conclusión: MySQL es una elección sólida y coherente con el modelo de datos relacional del sistema y la experiencia del equipo.

3.4. Madurez de las tecnologías aplicadas y escala esperada

Todas las tecnologías seleccionadas cuentan con una madurez probada:

Tecnología	Madurez
Java	Lenguaje con más de 25 años de desarrollo, utilizado en millones de proyectos empresariales.
Spring Boot	Framework líder en el ecosistema Java, con amplia adopción en la industria.
MySQL	Base de datos relacional más popular, con más de 25 años de desarrollo.
HTML + CSS + JavaScript	Tecnologías fundamentales de la web, con décadas de desarrollo y estabilidad.
TypeScript	Superset de JavaScript con adopción creciente, mantenido por Microsoft.
Spring Security	Estándar de seguridad en aplicaciones Spring, con amplia documentación y comunidad.
JWT	Estándar abierto para autenticación, ampliamente utilizado.

Escala esperada:

Para el MVP, el sistema estará orientado a talleres pequeños o medianos, con entre 1 y 10 empleados y un volumen de órdenes de 5 a 20 por día. El stack elegido es capaz de manejar esta escala sin problemas:

- Spring Boot ofrece excelente rendimiento para aplicaciones transaccionales.
- MySQL maneja sin problemas volúmenes moderados de datos.
- Los servicios de despliegue considerados ofrecen alternativas de bajo costo o adecuadas para proyectos educativos y de prueba, compatibles con las necesidades del MVP.

3.5. Despliegue

Se ha optado por un enfoque basado en servicios PaaS (Plataforma como Servicio) por las siguientes razones:

- **Reducción de complejidad operativa:** El equipo no debe gestionar servidores ni configurar entornos complejos. Puede enfocarse en el desarrollo.
- **Costos reducidos para un proyecto académico:** Se priorizarán plataformas que permitan realizar el despliegue con un costo bajo o que dispongan de opciones adecuadas para proyectos educativos y de prueba de manera gratuita.
- **Integración continua:** Se integran fácilmente con GitHub para despliegues automáticos.

Las plataformas propuestas son:

Componente	Plataforma	Justificación
Frontend	Vercel o Netlify	Especializadas en despliegue de sitios web estáticos. Integración sencilla con Git. Opciones adecuadas para proyectos de prueba y despliegues de bajo costo.
Backend (Java + Spring Boot)	Render o Railway	Soporte para aplicaciones Java (JAR). Despliegue sencillo desde Git. Opciones de despliegue compatibles con aplicaciones Java y adecuadas para proyectos académicos.
Base de Datos (MySQL)	Clever Cloud, Aiven o AWS RDS	Servicios administrados compatibles con MySQL que permiten disponer de una base de datos accesible desde el backend desplegado y reducen la complejidad de administración. .

Conclusión: Las alternativas consideradas permiten desplegar los principales componentes del sistema mediante servicios administrados, reduciendo la complejidad operativa. La selección definitiva de las plataformas se realizará considerando compatibilidad con el stack, disponibilidad del servicio y costos al momento del despliegue.

3.6. Costos de cambios y exploración

La elección de Java + Spring Boot + MySQL se apoya en la experiencia previa adquirida durante la carrera, la disponibilidad de documentación y la existencia de una amplia variedad de recursos de aprendizaje.

La incorporación de TypeScript como complemento de JavaScript no representa un cambio de paradigma, sino una extensión que aporta beneficios adicionales sin requerir un aprendizaje profundo.

Esto permite reducir la curva de aprendizaje, facilitar la resolución de problemas durante el desarrollo y concentrar los esfuerzos del equipo en la implementación de las funcionalidades propias del sistema.

No se desea experimentar con tecnologías nuevas, lo que minimiza el riesgo de descubrir limitaciones en etapas avanzadas del desarrollo.

Con base en la experiencia general del equipo y en los conocimientos desarrollados durante la carrera, definimos nuestro nivel de experiencia previa de la siguiente manera:

I

Tecnología	Nivel de conocimiento
Java	Intermedio
Spring Boot	Intermedio
MySQL	Intermedio
HTML + CSS + JavaScript	Intermedio
TypeScript	Básico-Intermedio

Propuesta y Plan de trabajo

1.1 Refinamiento de la Idea y Propuesta de Valor

La idea inicial del Sistema Integral de Gestión para Talleres y Servicios Técnicos llamado FixHub surgió como una solución al problema de la gestión fragmentada de reparaciones, donde la información se encuentra dispersa entre papel, planillas de cálculo, WhatsApp y llamadas telefónicas.

Para refinar la propuesta inicial e identificar con mayor claridad el problema, las necesidades de los usuarios y el alcance de la solución, se utilizó la inteligencia artificial como herramienta de apoyo durante el proceso de análisis.

1.2. Identificación de puntos de dolor

A partir de la interacción con la IA como herramienta de apoyo al análisis, se refinaron los siguientes puntos de dolor del usuario:

Punto de dolor	Descripción
Falta de visibilidad del estado	No existe una manera rápida de conocer en qué etapa se encuentra una orden de trabajo.
Información fragmentada	Clientes, equipos, órdenes, diagnósticos y presupuestos se registran en herramientas separadas.
Control de stock inconsistente	Los repuestos se consumen sin actualizar el inventario de manera sistemática.
Comunicación con el cliente	El cliente debe llamar o enviar mensajes para conocer el estado de su reparación.
Presupuestos sin registro formal	La aprobación del cliente se gestiona verbalmente o por mensaje, sin quedar registrada.
Falta de historial	No se conserva un registro estructurado de reparaciones por cliente o equipo.
Ausencia de indicadores	Sin datos sistematizados, es difícil obtener información para la toma de decisiones.

1.3. Definición del MVP

A partir de la interacción con la IA como herramienta de apoyo al análisis, se definió el siguiente alcance mínimo viable:

Funcionalidad	Descripción
Autenticación y roles	Autenticación e inicio de sesión con roles: Administrador, Recepcionista, Técnico y Cliente.
Gestión de clientes	Alta, edición y consulta de clientes.
Gestión de equipos	Alta, edición y consulta de equipos asociados a un cliente. Historial de reparaciones del equipo.
Órdenes de trabajo	Creación de órdenes, asignación de técnico, registro del problema inicial, cambio de estados y visualización del estado actual.
Diagnóstico	Registro de diagnóstico y observaciones técnicas.
Presupuestos	Registro de mano de obra, repuestos, cálculo automático del total, y aprobación/rechazo por parte del cliente.
Inventario y stock	Registro de repuestos, control de stock disponible, y consumo en reparaciones con actualización automática.
Historial	Registro de cambios importantes en cada orden de trabajo.
Seguimiento al cliente	Consulta del estado de su reparación y aprobación de presupuestos.
Dashboard básico	Órdenes pendientes, en reparación, listas para retirar, y stock bajo.

Comentario:

Esta definición del alcance permite abordar el problema central, demostrar el valor de la propuesta y mantener un conjunto de funcionalidades viable de acuerdo con los tiempos disponibles y la conformación del equipo de desarrollo, integrado por tres personas.

2. Análisis de Competencia y Diferenciación

Para el análisis preliminar se consideran como competidores directos las soluciones de software orientadas específicamente a la gestión de talleres y servicios técnicos, ya que buscan resolver necesidades similares para el mismo tipo de usuario. Como competidores indirectos se consideran alternativas como la gestión mediante papel y planillas de cálculo, los sistemas de gestión genéricos y los sistemas ERP, que pueden cubrir parcialmente estas necesidades pero no necesariamente están diseñados para este contexto específico.

A partir del análisis competitivo, se identificaron los siguientes elementos diferenciadores del sistema FixHub:

Diferenciador	Descripción
Ciclo completo integrado	El sistema cubre todo el ciclo de trabajo: recepción → diagnóstico → presupuesto → aprobación → reparación → entrega.
Control de stock integrado	El inventario se actualiza automáticamente al consumir repuestos en una reparación.
Seguimiento al cliente	El cliente puede consultar el estado de su reparación y aprobar presupuestos sin necesidad de llamar.
Historial por equipo y cliente	Se conserva un registro estructurado de todas las reparaciones realizadas.
Roles y permisos	Diferentes niveles de acceso según el rol (Administrador, Recepcionista, Técnico, Cliente).
Facilidad de uso	Orientado a talleres y servicios técnicos pequeños y medianos, con una interfaz enfocada en las funcionalidades necesarias para la gestión operativa cotidiana.

2.2. Matriz competitiva

Con ayuda de la IA como herramienta de apoyo, se realizó un análisis comparativo preliminar de distintas alternativas utilizadas para la gestión de talleres y servicios técnicos, considerando herramientas manuales, software de gestión genérico y sistemas ERP.

Variable	Sistema propuesto	Gestión en papel/planillas	Software de gestión genérico	Sistemas ERP
Ciclo completo integrado	✓ Sí	✗ No	⚠ Parcial	✓ Sí
Control de stock	✓ Sí	✗ No	⚠ Parcial	✓ Sí
Seguimiento al cliente	✓ Sí	✗ No	✗ No	⚠ Parcial
Historial por equipo	✓ Sí	✗ No	⚠ Parcial	✓ Sí
Roles y permisos	✓ Sí	✗ No	⚠ Parcial	✓ Sí
Costo	Bajo (MVP)	Bajo	Medio	Alto
Complejidad de uso	Baja	Baja	Media	Alta
Orientado a talleres pequeños	✓ Sí	✓ Sí	✗ No	✗ No

3. Objetivo general y específicos

- Objetivo general:

Desarrollar el MVP de FixHub, un Sistema Integral de Gestión para Talleres y Servicios Técnicos que permita administrar el ciclo completo de reparaciones, desde la recepción del equipo hasta su entrega al cliente, centralizando la información y automatizando tareas manuales.

- Objetivos específicos:

- Implementar un sistema de autenticación con roles (Administrador, Recepcionista, Técnico, Cliente) que permita controlar el acceso a las funcionalidades según el perfil del usuario.

- Desarrollar un módulo de gestión de clientes y equipos que permita registrar, consultar y mantener el historial de reparaciones.
- Implementar el ciclo de vida de una orden de trabajo, incluyendo su creación, asignación de técnico, cambios de estado y registro del problema inicial.
- Desarrollar un módulo de diagnóstico que permita registrar observaciones técnicas asociadas a una orden.
- Implementar un módulo de presupuestos que calcule automáticamente el total (mano de obra + repuestos) y permita la aprobación/rechazo por parte del cliente.
- Desarrollar un módulo de inventario y stock que registre repuestos, controle el stock disponible y actualice automáticamente el inventario al consumir repuestos en una reparación.
- Implementar un sistema de historial que registre los cambios importantes en cada orden de trabajo.
- Implementar una funcionalidad de seguimiento que permita al cliente consultar el estado de su reparación y gestionar la aprobación o rechazo de presupuestos.
- Desarrollar un dashboard básico que muestre indicadores clave como órdenes pendientes, en reparación, listas para retirar y stock bajo.
- Desplegar la aplicación en un entorno accesible para su prueba y presentación.

3.1 Alcances y no alcances

En este cuadro mostramos en conjunto los alcances y no alcances del sistema propuesto:

Funcionalidad	Alcance (MVP)	No alcance (MVP)
Autenticación y roles	Autenticación e inicio de sesión. Roles: Administrador, Recepcionista, Técnico y Cliente.	-
Gestión de clientes	Alta, edición y consulta de clientes.	-
Gestión de equipos	Alta, edición y consulta de equipos asociados a clientes.	-
Historial de reparaciones por equipo	Visualización del historial de reparaciones de un equipo.	-
Órdenes de trabajo	Creación, asignación de técnico, cambio de estados, visualización del estado actual.	-

Funcionalidad	Alcance (MVP)	No alcance (MVP)
Diagnóstico	Registro de diagnóstico y observaciones técnicas.	-
Presupuestos	Registro de mano de obra y repuestos, cálculo automático del total, aprobación/rechazo por el cliente.	-
Inventario y stock	Registro de repuestos, control de stock disponible, consumo con actualización automática.	-
Historial de cambios	Registro de cambios importantes en cada orden de trabajo.	-
Seguimiento al cliente	Consulta de estado de su reparación y aprobación de presupuestos.	-
Dashboard básico	Indicadores: órdenes pendientes, en reparación, listas para retirar, stock bajo.	-
Interfaz web responsive	Accesible desde computadoras y dispositivos móviles.	-
Pagos online	-	No incluido. No es esencial para la gestión del taller.
Facturación electrónica AFIP/ARCA	-	No incluido. Complejidad regulatoria, no esencial para el MVP.
Integración con WhatsApp Business	-	No incluido. Requiere integración con APIs externas.
Aplicación móvil nativa	-	No incluido. Se prioriza la versión web responsive.
Geolocalización	-	No incluido. No es esencial.
Inteligencia artificial	-	No incluido. No es esencial para el MVP.
Múltiples sucursales	-	No incluido. El MVP está orientado a talleres de una sola ubicación.
Contabilidad completa	-	No incluido. Se prioriza la gestión operativa.
Compras automáticas a proveedores	-	No incluido. No es esencial para el MVP.

Funcionalidad	Alcance (MVP)	No alcance (MVP)
Marketplace de repuestos	-	No incluido. No es esencial para el MVP.

3.2. Criterios de éxito del MVP

Para considerar que el MVP de FixHub cumple con su propósito, se establecen los siguientes criterios mínimos de éxito:

- El sistema permite registrar clientes y equipos y mantener correctamente asociada su información.
- Es posible crear una orden de trabajo, asignarla a un técnico y gestionar sus cambios de estado respetando el flujo definido.
- El técnico puede registrar un diagnóstico asociado a una orden de trabajo.
- El sistema permite generar un presupuesto con mano de obra y repuestos, calcular su total y registrar la aprobación o rechazo por parte del cliente.
- Una orden que requiera presupuesto no puede avanzar a reparación mientras este no haya sido aprobado.
- El consumo de repuestos durante una reparación actualiza correctamente el stock disponible.
- Los cambios importantes realizados sobre una orden quedan registrados en su historial.
- El cliente puede consultar el estado de su reparación y gestionar la aprobación o rechazo de un presupuesto.
- Los distintos usuarios acceden únicamente a las funcionalidades correspondientes a sus roles y permisos.
- El dashboard permite visualizar información básica de gestión, como órdenes pendientes, en reparación, listas para retirar y repuestos con stock bajo.
- La aplicación puede ser desplegada en un entorno accesible para realizar pruebas y presentar el funcionamiento del MVP.

3.3. Entregables por etapa

La planificación se organiza en etapas internas de trabajo y se ajustará a los hitos y fechas de entrega establecidos por la cátedra. Las duraciones indicadas son estimativas y podrán adaptarse de acuerdo con el avance del proyecto, manteniendo como referencia las entregas oficiales del TFI.

Etapa	Duración	Entregables
Etapa 1: Planificación y diseño	Semanas 1-2	Documentación inicial. Diseño de UI/UX. Prototipo de interfaz. Definición de arquitectura y stack tecnológico. Modelado de base de datos.
Etapa 2: Desarrollo del backend	Semanas 3-5	API REST funcional con Spring Boot. Autenticación y roles con Spring Security + JWT. Entidades y repositorios con Spring Data JPA. Lógica de negocio: órdenes, estados, presupuestos, stock.
Etapa 3: Desarrollo del frontend	Semanas 4-6	Interfaz de usuario con HTML + CSS + JavaScript + TypeScript. Pantallas de login, dashboard, gestión de clientes, equipos, órdenes, presupuestos, stock. Consumo de la API REST.
Etapa 4: Integración y pruebas	Semanas 6-7	Integración frontend-backend. Pruebas funcionales y de integración. Validación de los principales flujos de usuario. Corrección de errores.
Etapa 5: Despliegue y presentación	Semana 8	Despliegue en plataformas PaaS. Documentación técnica. Video demo. Presentación final.

Esta planificación deberá permitir cumplir con la primera entrega de propuesta y repositorio, la segunda entrega correspondiente al diseño y definición de módulos, y la entrega final con el sistema completo, documentación, despliegue y video demostrativo, de acuerdo con el cronograma oficial de la materia.

3.4. Riesgos iniciales y mitigaciones

Riesgo	Mitigación
Complejidad de la lógica de negocio	Definir claramente las reglas de negocio antes del desarrollo. Realizar pruebas unitarias de los casos críticos.
Control de stock y transacciones	Utilizar transacciones ACID en las operaciones de consumo de repuestos. Implementar pruebas de integridad.
Seguridad y roles	Utilizar Spring Security correctamente configurado. Probar los diferentes roles y permisos.

Riesgo	Mitigación
Desempeño de la API	Optimizar consultas JPA. Utilizar paginación en listados extensos.
Disponibilidad del equipo	Establecer reuniones periódicas de sincronización y definir responsabilidades claras.
Despliegue en plataformas PaaS	Verificar que las plataformas elegidas soporten aplicaciones Java y MySQL. Realizar pruebas de despliegue tempranas.
Protección de datos personales y acceso no autorizado	Implementar autenticación segura, control de acceso según roles, almacenamiento seguro de contraseñas y restricciones de acceso a la información según las responsabilidades de cada usuario.
Pérdida de información	Utilizar una base de datos persistente, respaldo y manejo adecuado de errores

4. Resumen de viabilidad técnica, operativa y temporal.

Dimensión	Evaluación	Justificación
Técnica	✓ Viable	Stack conocido, bibliotecas maduras, arquitectura definida.
Operativa	✓ Viable	Problema observado en un caso real y con potencial de aplicación en otros servicios técnicos. La aceptación por parte de otros usuarios será validada mediante relevamiento.
Temporal	✓ Viable	Alcance acotado al MVP, planificación por etapas ajustada al cronograma oficial de la materia y equipo de desarrollo integrado por tres personas.

5. Conclusiones finales

El Sistema Integral de Gestión FixHub ha sido definido y evaluado en sus tres dimensiones principales:

Identificación del problema: Se ha identificado una problemática real relacionada con la gestión fragmentada de reparaciones en talleres y servicios técnicos, donde la información puede encontrarse dispersa entre papel, planillas de cálculo, WhatsApp y llamadas telefónicas. El análisis parte de una situación concreta observada en un servicio técnico y se plantea como una problemática potencialmente aplicable a otros rubros. Esta situación genera pérdida de tiempo, falta de visibilidad del estado de las órdenes, control de stock inconsistente y comunicación deficiente con el cliente.

Definición del stack tecnológico: Se ha seleccionado un stack coherente con las necesidades del proyecto y la experiencia del equipo: Java + Spring Boot para el backend con lógica de negocio compleja, MySQL como base de datos relacional, y HTML + CSS + JavaScript + TypeScript para un frontend funcional con tipado estático.

Propuesta y viabilidad: Se ha refinado la propuesta de valor utilizando la inteligencia artificial como herramienta de apoyo al análisis, se ha definido el alcance del MVP con funcionalidades esenciales y se ha establecido un plan de trabajo acorde con los tiempos y recursos disponibles. A partir de este análisis, se considera que el desarrollo del sistema resulta viable desde las perspectivas técnica, operativa y temporal.